

CENG493 Turkish Legal RAG Progress Report

Selected Models

For the full neural version, we selected the same LLM for Base RAG and Fine-tuned RAG:

Qwen/Qwen2.5-3B-Instruct. It is small enough for LoRA/QLoRA-style supervised fine-tuning, has strong instruction-following behavior, and supports multilingual legal question answering better than a small English-only model. The embedding backbone is **intfloat/multilingual-e5-base**, because the task is Turkish retrieval and the model is a general multilingual sentence embedding model. The reranker backbone is **BAAI/bge-reranker-v2-m3**, because it is multilingual and designed for cross-encoder reranking.

The repository contains two runnable paths: a standard-library fallback RAG pipeline for quick reproducible checks, and a GPU training path using PyTorch, Transformers, PEFT, SentenceTransformers, and bitsandbytes.

References checked on 30 May 2026:

- **Qwen/Qwen2.5-3B-Instruct**: <https://huggingface.co/Qwen/Qwen2.5-3B-Instruct>
- **intfloat/multilingual-e5-base**: <https://huggingface.co/intfloat/multilingual-e5-base>
- **BAAI/bge-reranker-v2-m3**: <https://huggingface.co/BAAI/bge-reranker-v2-m3>

Finetuning Strategy

The planned full system has three adapted components:

1. Embedding model adaptation using **embedding.jsonl**, where each row contains a query, positive passage, negative passage, and document identifiers. The intended loss is a contrastive loss with in-batch negatives plus the provided hard negative. 2. Reranker adaptation using **reranker.jsonl**, where each row contains a query, candidate passage, and binary relevance label. The intended loss is binary cross entropy over query-passage pairs. 3. LLM supervised fine-tuning using **llm.jsonl**, where each row is a chat example teaching the model to answer only from supplied context and always cite the source.

The same selected LLM is used for both Base RAG and Fine-tuned RAG. Base RAG uses the unadapted retrieval/generation policy; Fine-tuned RAG uses adapted retrieval weights, adapted reranking weights, and the citation-grounded answer style learned from the SFT examples.

Dataset Preparation So Far

We use **Datasets_Ceng493_legal_rag**, not the two datasets mentioned in the assignment PDF. The current dataset contains:

- **corpus.jsonl**: 7,579 legal text chunks with ids, titles, text, metadata, category, citation labels, and source locations.
- **embedding.jsonl**: 2,059 query-positive-negative training triples.
- **reranker.jsonl**: 6,752 query-candidate-label examples.
- **llm.jsonl**: 13,758 grounded chat examples for SFT.
- **gold_benchmark.json**: 240 benchmark questions with verified answers and relevant source ids.
- **rag_eval.json**: 1,000 retrieval-oriented evaluation questions with gold chunk ids.

Data preparation already separates training examples from benchmark examples through the dataset metadata flags. The code preserves ids and citation labels so evaluator-provided custom benchmarks can measure retrieval and citation correctness.

Tests and Evaluation Plan

The runnable evaluation compares Base RAG and Fine-tuned RAG on the same benchmark. If gold document ids are available, we report Recall@1, Recall@3, Recall@5, MRR@5, citation hit rate, answer token F1, and grounded answer rate. These metrics are appropriate because the assignment evaluates both retrieval quality and final answer grounding.

Ablation study rows are included:

- Base RAG
- Adapted retriever only
- Adapted reranker only
- Adapted answer style only
- Full Fine-tuned RAG

The system also supports evaluator-provided custom document collections through **--custom-docs**. The evaluator can provide their own corpus and ask questions over it without changing the code.

GPU Training Completed

The final GPU runs were conducted on an NVIDIA GeForce RTX 4060 with 8 GB VRAM.

- Embedding fine-tune: **intfloat/multilingual-e5-base**, 300 GPU steps, average triplet loss 0.0561, saved to **artifacts/gpu/embedding_model**.
- Reranker fine-tune: **BAAI/bge-reranker-v2-m3**, 30 GPU optimizer steps, average BCE loss 0.3735, saved to **artifacts/gpu/reranker_model**.
- LLM LoRA/QLoRA SFT: **Qwen/Qwen2.5-3B-Instruct**, 60 GPU optimizer steps, 14,966,784 trainable LoRA parameters, average LM loss 0.9744, saved to **artifacts/gpu/llm_lora**.

Current Evaluation Results

Full lightweight RAG evaluation on the 240-item gold benchmark:

System	Recall@1	Recall@3	Recall@5	MRR@5	Answer F1	Citation Hit
Base RAG	0.875	0.950	0.971	0.912	0.395	0.808
Fine-tuned RAG adapter pipeline	0.879	0.942	0.971	0.912	0.398	0.800

GPU neural embedding retrieval on the first 100 gold benchmark questions:

Embedding	Recall@1	Recall@3	Recall@5	Recall@10	MRR@10
Base multilingual-e5-base	0.620	0.750	0.830	0.880	0.708
Fine-tuned embedding checkpoint	0.520	0.680	0.700	0.740	0.604

GPU neural embedding retrieval on the first 200 **rag_eval** questions:

Embedding	Recall@1	Recall@3	Recall@5	Recall@10	MRR@10
Base multilingual-e5-base	0.865	0.960	0.970	0.990	0.915
Fine-tuned embedding checkpoint	0.855	0.940	0.970	0.995	0.905

The current fine-tuned embedding checkpoint does not consistently beat the base embedding. This is an important result rather than something to hide: the short GPU run is enough to prove the training pipeline and produce a checkpoint, but not yet enough for a validated improvement. Next tuning steps are lower learning rate, validation-based

checkpoint selection, longer training, and mixing **rag_eval**-style questions into the embedding training distribution.